



软件系统设计复习

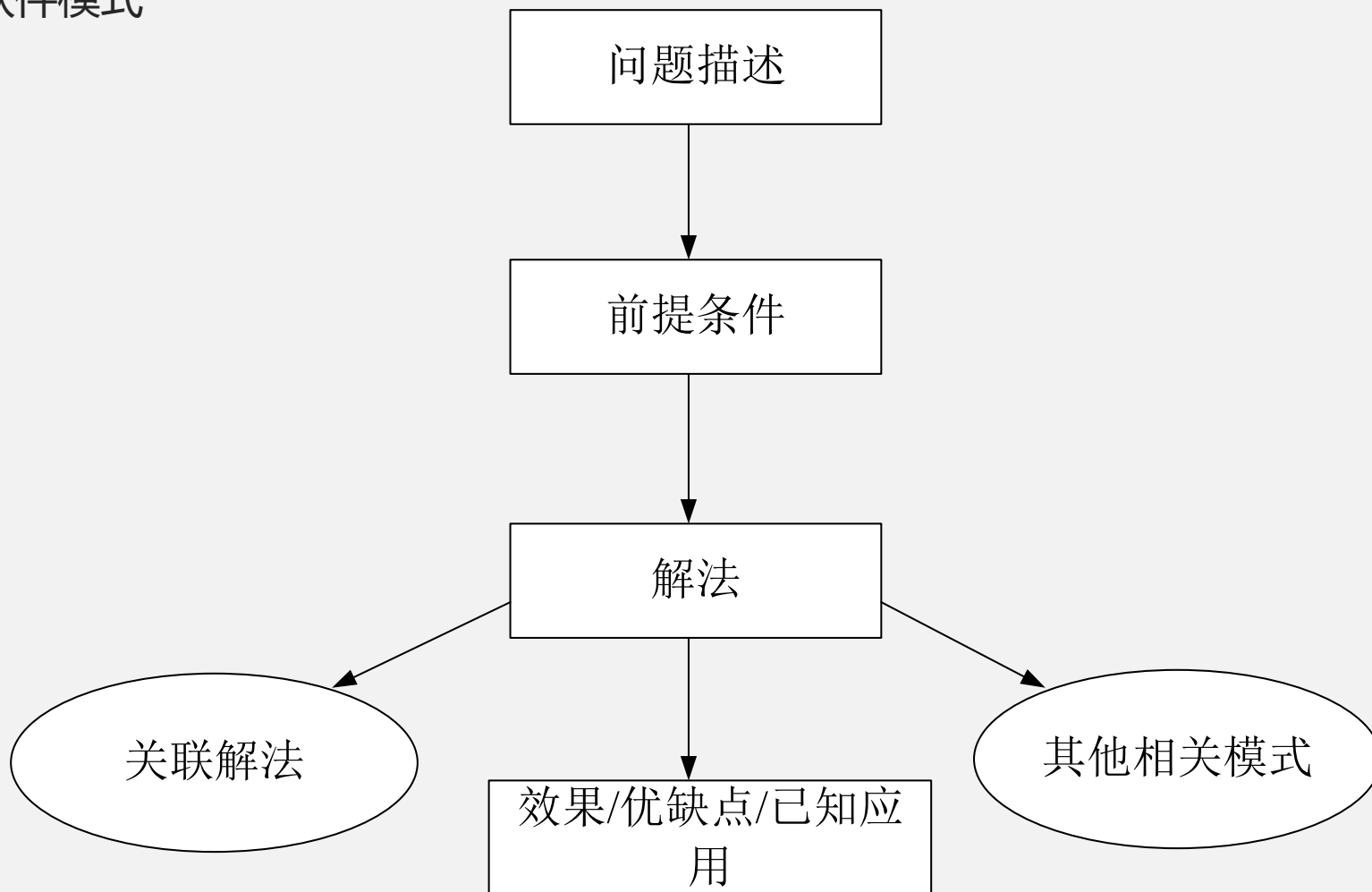
软件模式

- 软件模式

- 软件模式是将模式的一般概念应用于软件开发领域，即软件开发的总体指导思路或参照样板。软件模式并非仅限于设计模式，还包括架构模式、分析模式和过程模式等，实际上，在软件生存期的每一个阶段都存在着一些被认同的模式。
- 软件模式可以认为是对软件开发这一特定“问题”的“解法”的某种统一表示，软件模式等于一定条件下的出现的问题以及解法。软件模式的基础结构由4个部分构成：问题描述、前提条件（环境或约束条件）、解法和效果。

软件模式

- 软件模式



软件模式

- 软件模式
 - 软件模式与具体的应用领域无关，在模式发现过程中需要遵循大三律(Rule of Three)，即只有经过三个以上不同类型（或不同领域）的系统的校验，一个解决方案才能从候选模式升格为模式。

设计模式的定义

- 设计模式的定义
 - 设计模式(Design Pattern)是一套被反复使用、多数人知晓的、经过分类编目的、代码设计经验的总结，使用设计模式是为了可重用代码、让代码更容易被他人理解、保证代码可靠性。

设计模式的基本要素

- 设计模式的基本要素
 - 设计模式一般有如下几个基本要素：模式名称、问题、目的、解决方案、效果、实例代码和相关设计模式，其中的关键元素包括以下四个方面：
 - 模式名称 (Pattern name)
 - 问题 (Problem)
 - 解决方案 (Solution)
 - 效果 (Consequences)

设计模式的分类

• 设计模式的分类

- 根据其目的（模式是用来做什么的）可分为创建型(Creational)，结构型(Structural)和行为型(Behavioral)三种：
 - 创建型模式主要用于创建对象。
 - 结构型模式主要用于处理类或对象的组合。
 - 行为型模式主要用于描述对类或对象怎样交互和怎样分配职责。

设计模式的分类

• 设计模式的分类

- 根据范围，即模式主要是用于处理类之间关系还是处理对象之间的关系，可分为类模式和对象模式两种：
 - 类模式处理类和子类之间的关系，这些关系通过继承建立，在编译时刻就被确定下来，是属于静态的。
 - 对象模式处理对象间的关系，这些关系在运行时刻变化，更具动态性。

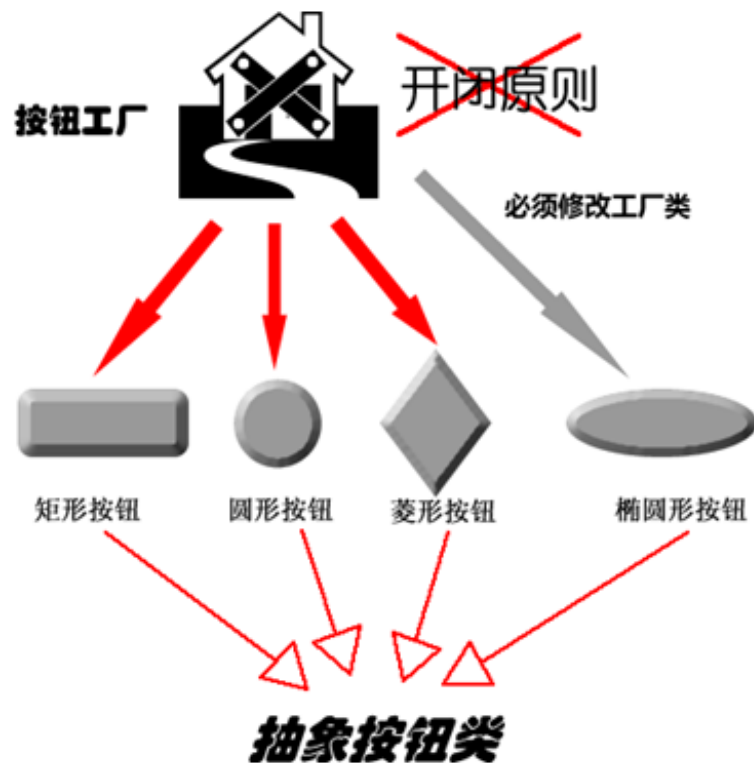
设计模式的分类

范围\目的	创建型模式	结构型模式	行为型模式
类模式	工厂方法模式	（类）适配器模式	模板方法模式
对象模式	抽象工厂模式 建造者模式 原型模式 单例模式	（对象）适配器模式 桥接模式 组合模式 装饰模式 外观模式 享元模式 代理模式	命令模式 迭代器模式 中介者模式 观察者模式 状态模式 策略模式

工厂方法模式

• 模式动机

- 考虑这样一个系统，按钮工厂类可以返回一个具体的按钮实例，如圆形按钮、矩形按钮、菱形按钮等。在这个系统中，如果需要增加一种新类型的按钮，如椭圆形按钮，那么除了增加一个新的具体产品类之外，还需要修改工厂类的代码，这就使得整个设计在一定程度上违反了“开闭原则”。

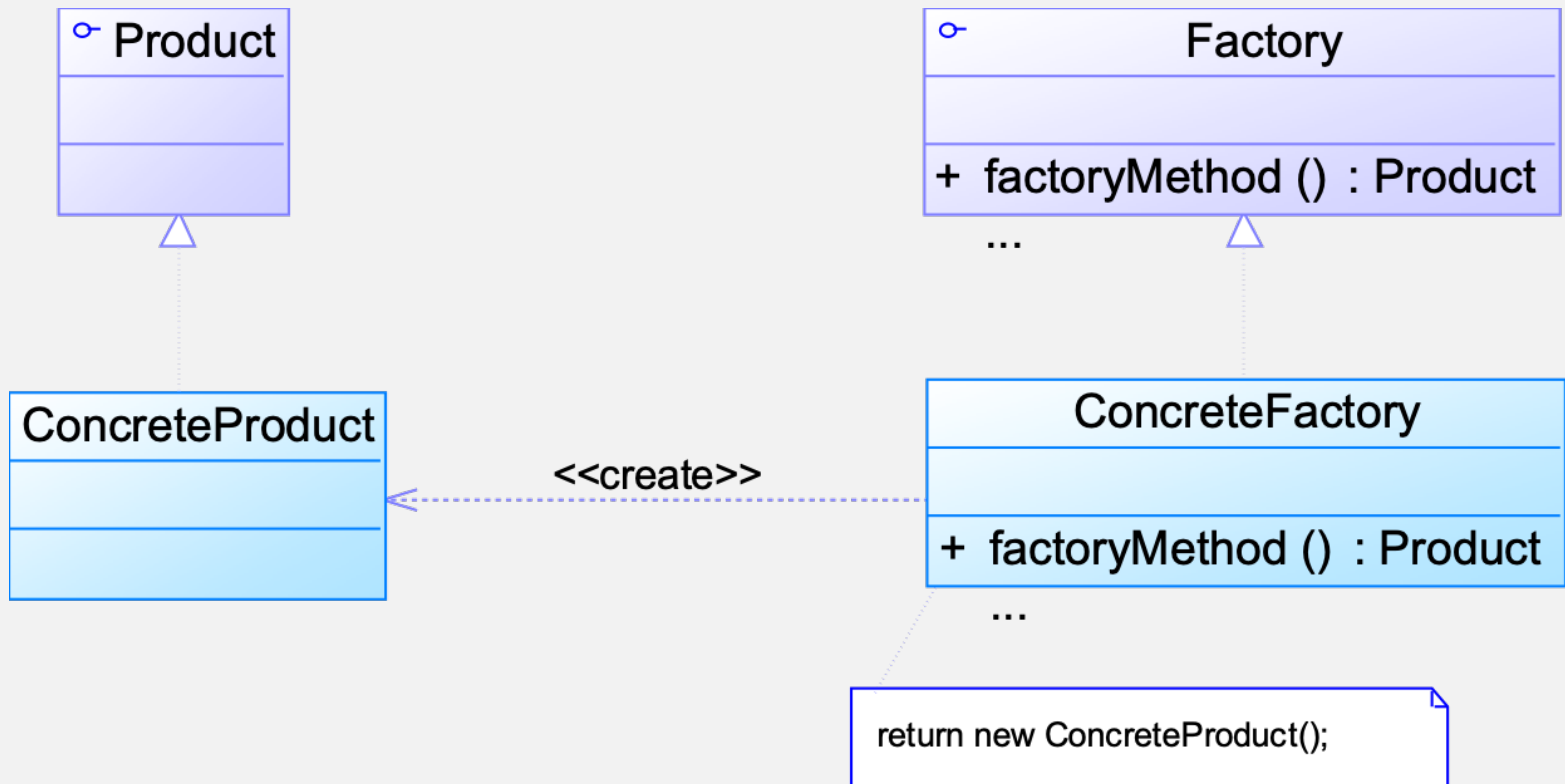


工厂方法模式

- 模式定义
 - 工厂方法模式(Factory Method Pattern)又称为工厂模式，也叫**虚拟构造器(Virtual Constructor)模式**或者**多态工厂(Polymorphic Factory)模式**，它属于类创建型模式。在工厂方法模式中，工厂父类负责定义创建产品对象的公共接口，而工厂子类则负责生成具体的产品对象，这样做的目的是将产品类的实例化操作延迟到工厂子类中完成，即通过工厂子类来确定究竟应该实例化哪一个具体产品类。

工厂方法模式

- 模式结构



抽象工厂模式

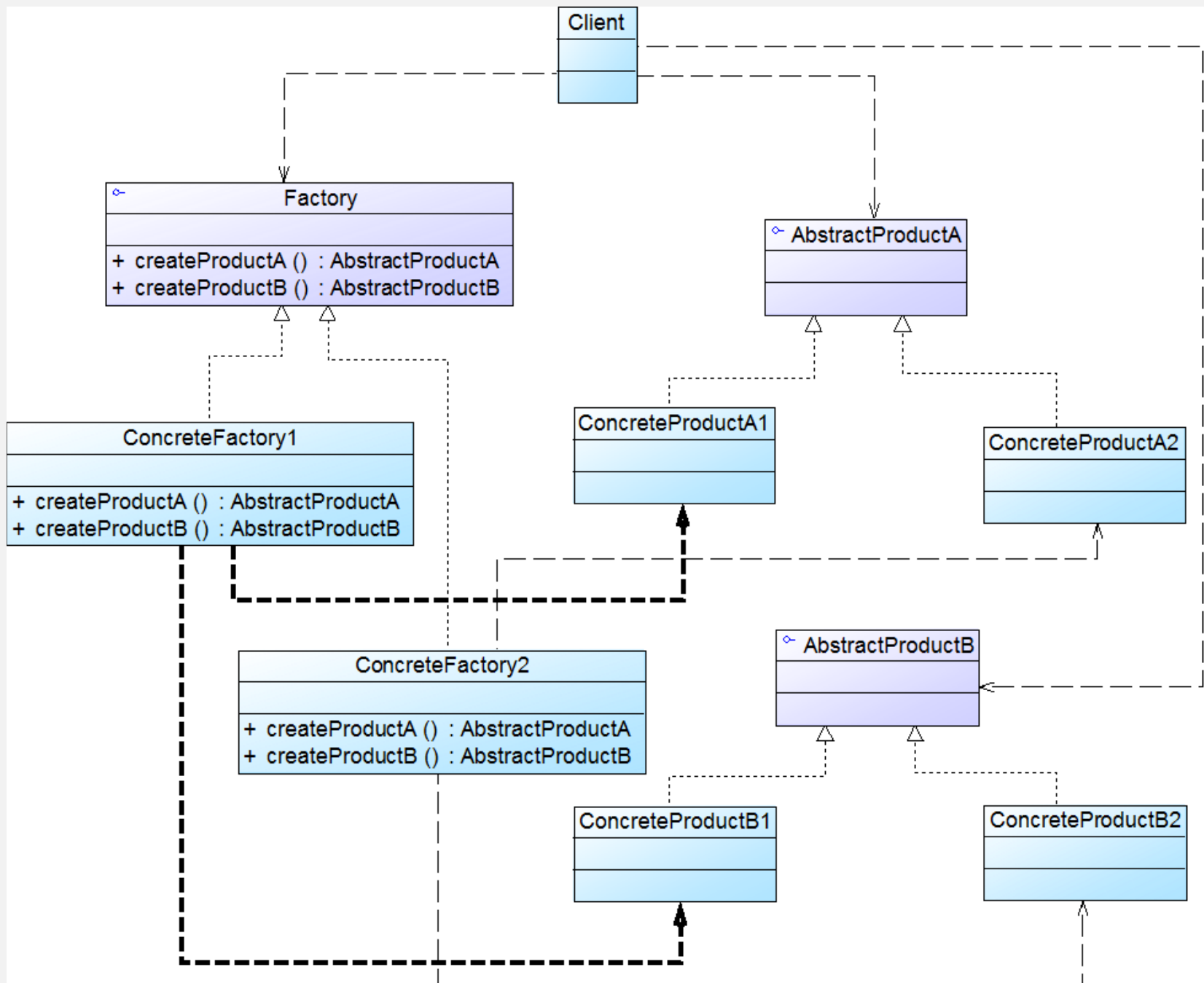
在工厂方法模式中具体工厂负责生产具体的产品，每一个具体工厂对应一种具体产品，工厂方法也具有唯一性，一般情况下，一个具体工厂中只有一个工厂方法或者一组重载的工厂方法。但是有时候我们需要一个工厂可以提供多个产品对象，而不是单一的产品对象。

抽象工厂模式

抽象工厂模式(Abstract Factory Pattern): 提供一个创建一系列相关或相互依赖对象的接口, 而无须指定它们具体的类。抽象工厂模式又称为Kit模式, 属于对象创建型模式。

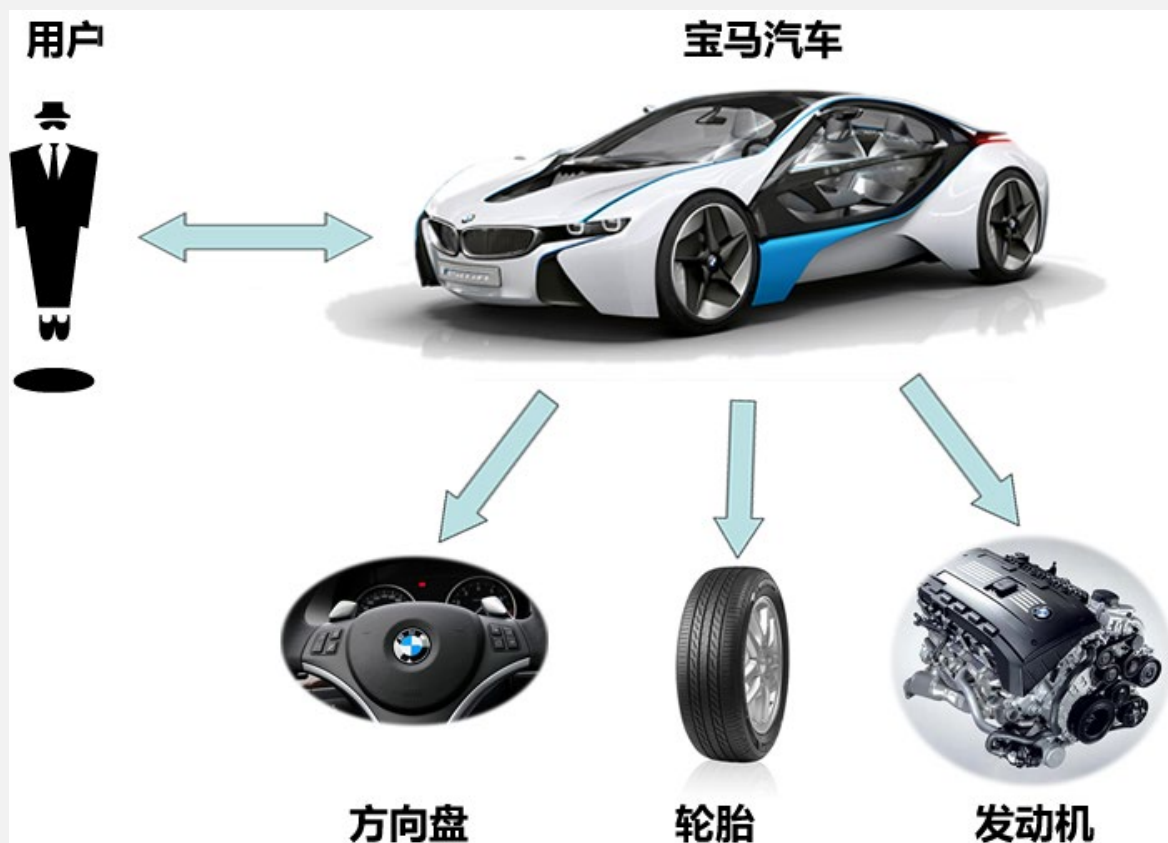
Abstract Factory Pattern: Provide an interface for creating families of related or dependent objects without specifying their concrete classes.

Frequency of use: high



建造者模式

- 模式动机

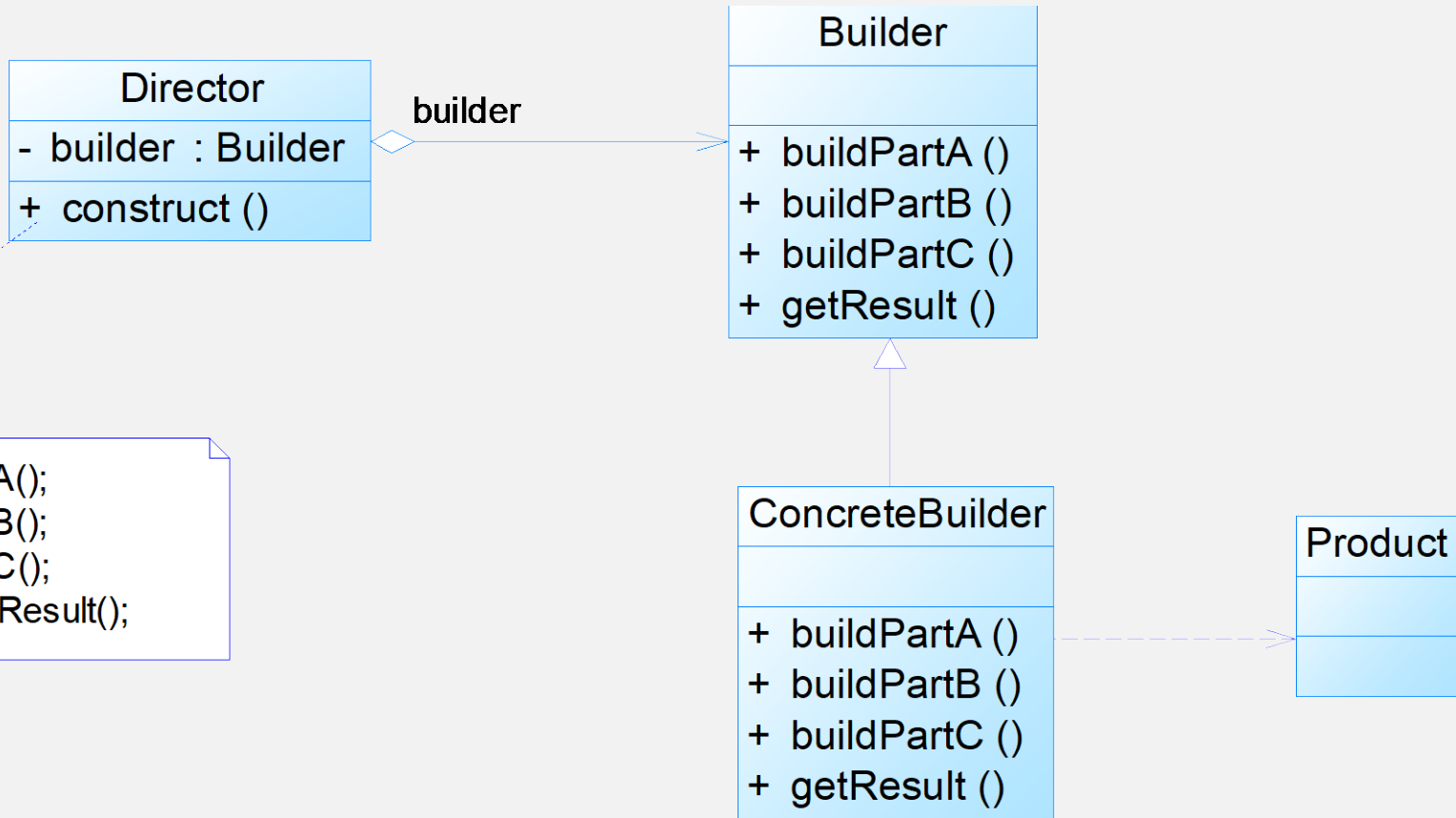


建造者模式

- 模式定义
 - 建造者模式(Builder Pattern): 将一个复杂对象的构建与它的表示分离, 使得同样的构建过程可以创建不同的表示。
 - Builder Pattern: Separate the construction of a complex object from its representation so that the same construction process can create different representations.
 - 建造者模式是一步一步创建一个复杂的对象, 它允许用户只通过指定复杂对象的类型和内容就可以构建它们, 用户不需要知道内部的具体构建细节。建造者模式属于对象创建型模式。根据中文翻译的不同, 建造者模式又可以称为生成器模式。

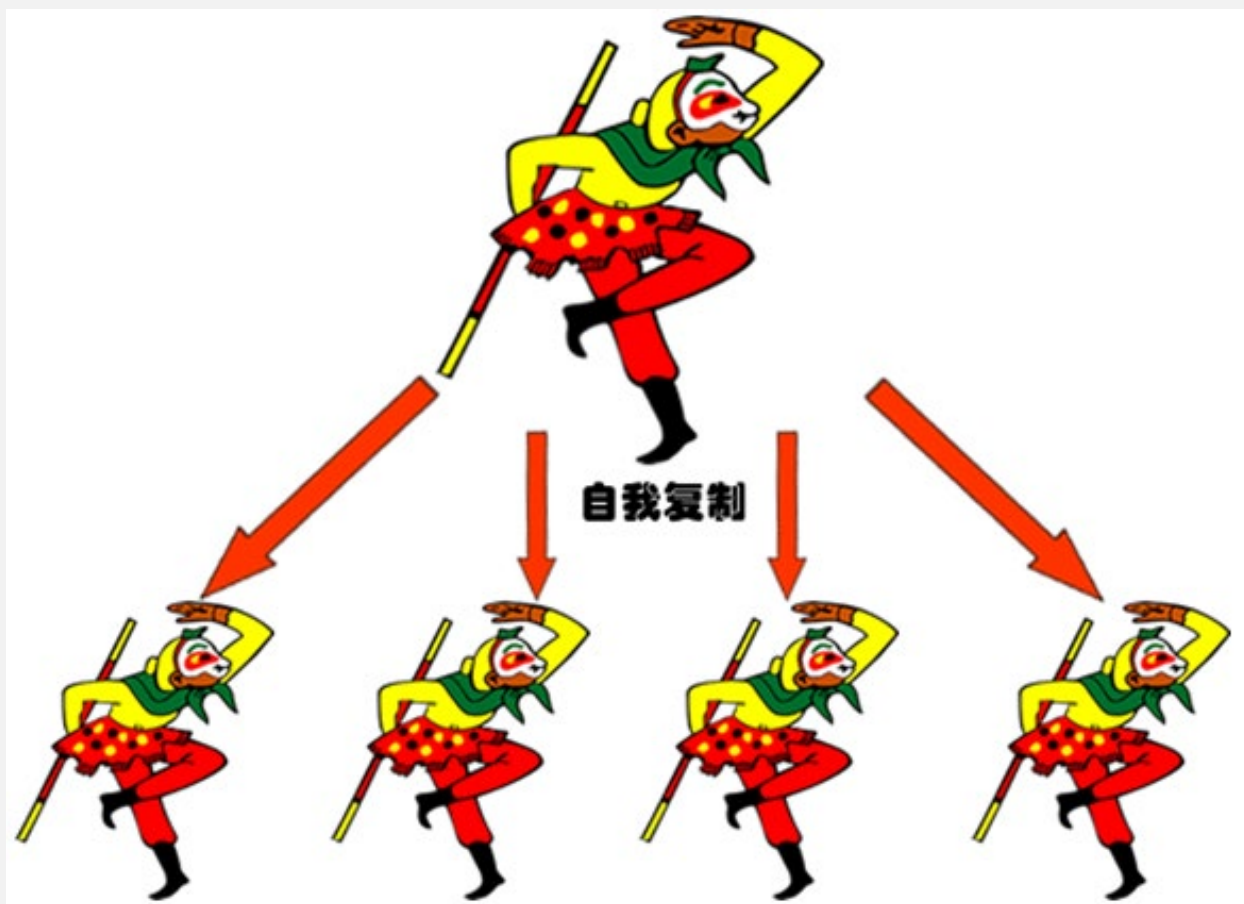
建造者模式

- 模式结构



原型模式

- 模式动机

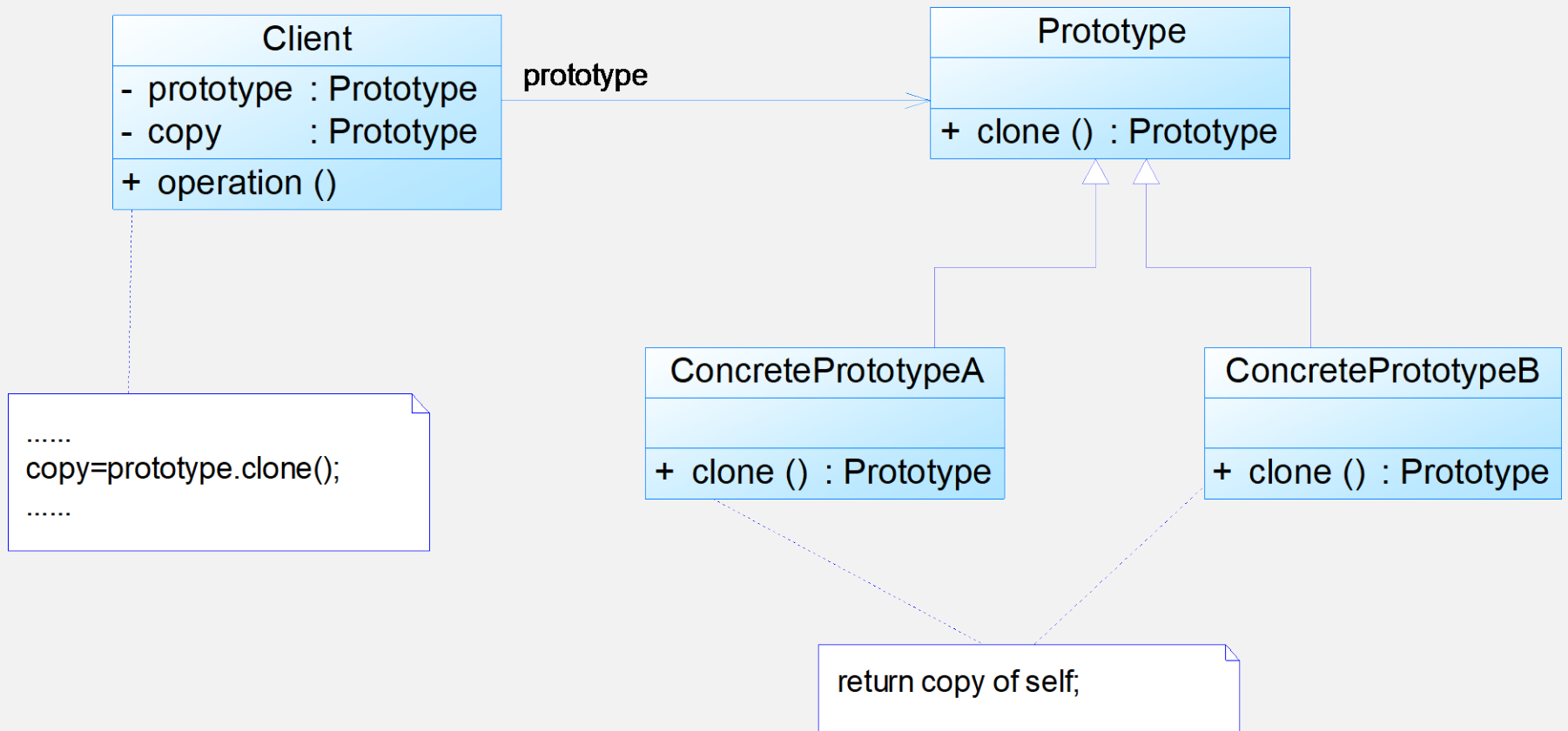


原型模式

- 模式定义
 - 原型模式(Prototype Pattern): 原型模式是一种对象创建型模式, 用原型实例指定创建对象的种类, 并且通过复制这些原型创建新的对象。原型模式允许一个对象再创建另外一个可定制的对象, 无须知道任何创建的细节。
 - **Prototype Pattern:** Specify the kind of objects to create using a prototypical instance, and create new objects by copying this prototype.
 - 原型模式的基本工作原理是通过将一个原型对象传给那个要发动创建的对象, 这个要发动创建的对象通过请求原型对象拷贝原型自己来实现创建过程。

原型模式

- 模式结构



单例模式

模式名称：
单例模式。

问题：
系统需要一个类的唯一实例，
以保证全局状态的一致性。

解决方案：
私有构造方法，提供一个静态
方法，在多线程环境下，使用
静态内部类优化性能

效果：
保证数据一致，减少实例，
增加耦合，难扩展，性能瓶颈

```
public class Singleton {  
    private static Singleton uniqueInstance;  
  
    // other useful instance variables here  
  
    private Singleton() {}  
  
    public static Singleton getInstance() {  
        if (uniqueInstance == null) {  
            uniqueInstance = new Singleton();  
        }  
        return uniqueInstance;  
    }  
  
    // other useful methods here  
}
```

We have a static
variable to hold our
one instance of the
class Singleton.

Our constructor is
declared private;
only Singleton can
instantiate this class!

The getInstance()
method gives us a way
to instantiate the class
and also to return an
instance of it.

Of course, Singleton is
a normal class; it has
other useful instance
variables and methods.

适配器模式

- 模式动机

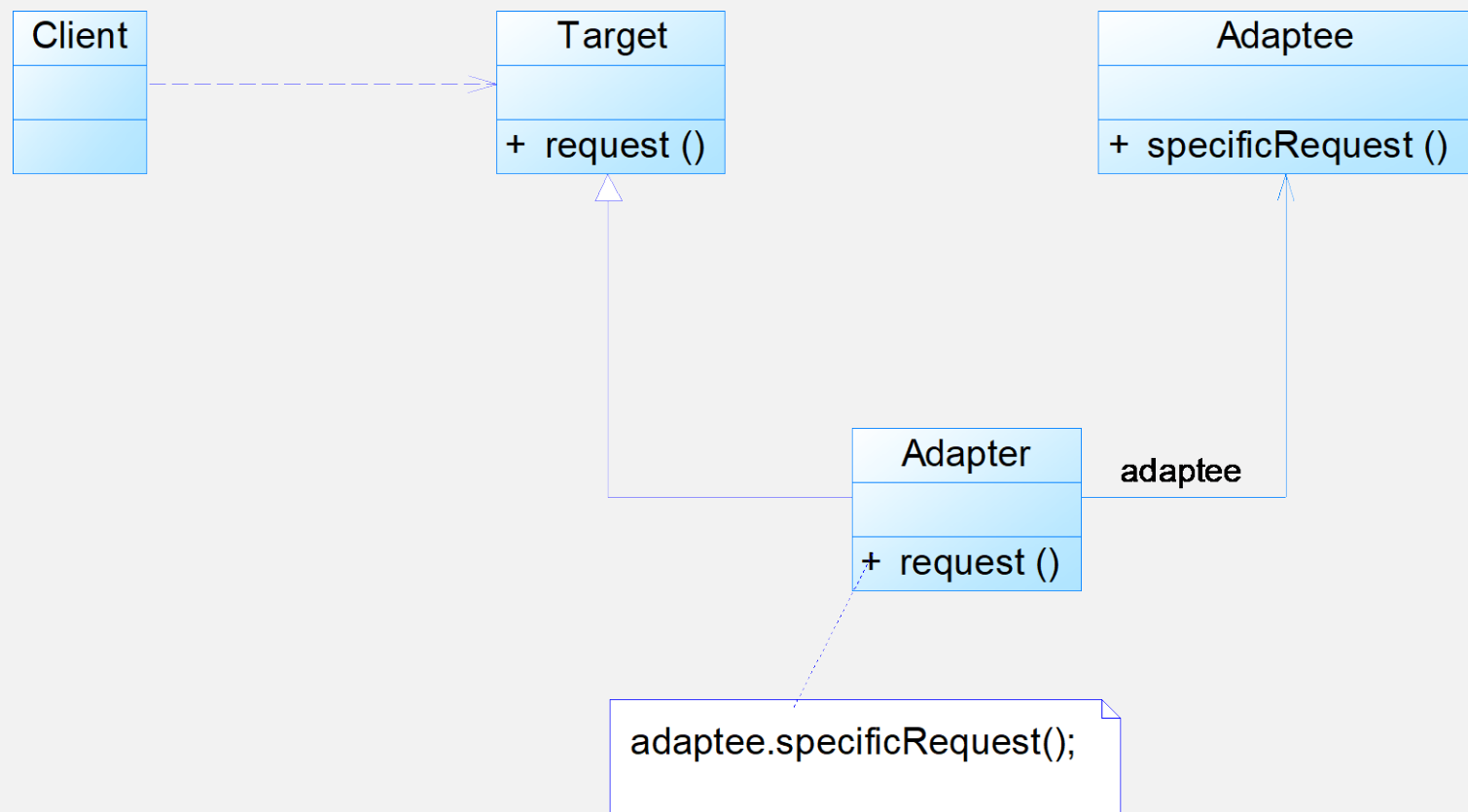


适配器模式

- 模式定义
 - 适配器模式(Adapter Pattern)： 将一个接口转换成客户希望的另一个接口，适配器模式使接口不兼容的那些类可以一起工作，其别名为包装器(Wrapper)。适配器模式既可以作为类结构型模式，也可以作为对象结构型模式。
 - Adapter Pattern: Convert the interface of a class into another interface clients expect. Adapter lets classes work together that couldn't otherwise because of incompatible interfaces.

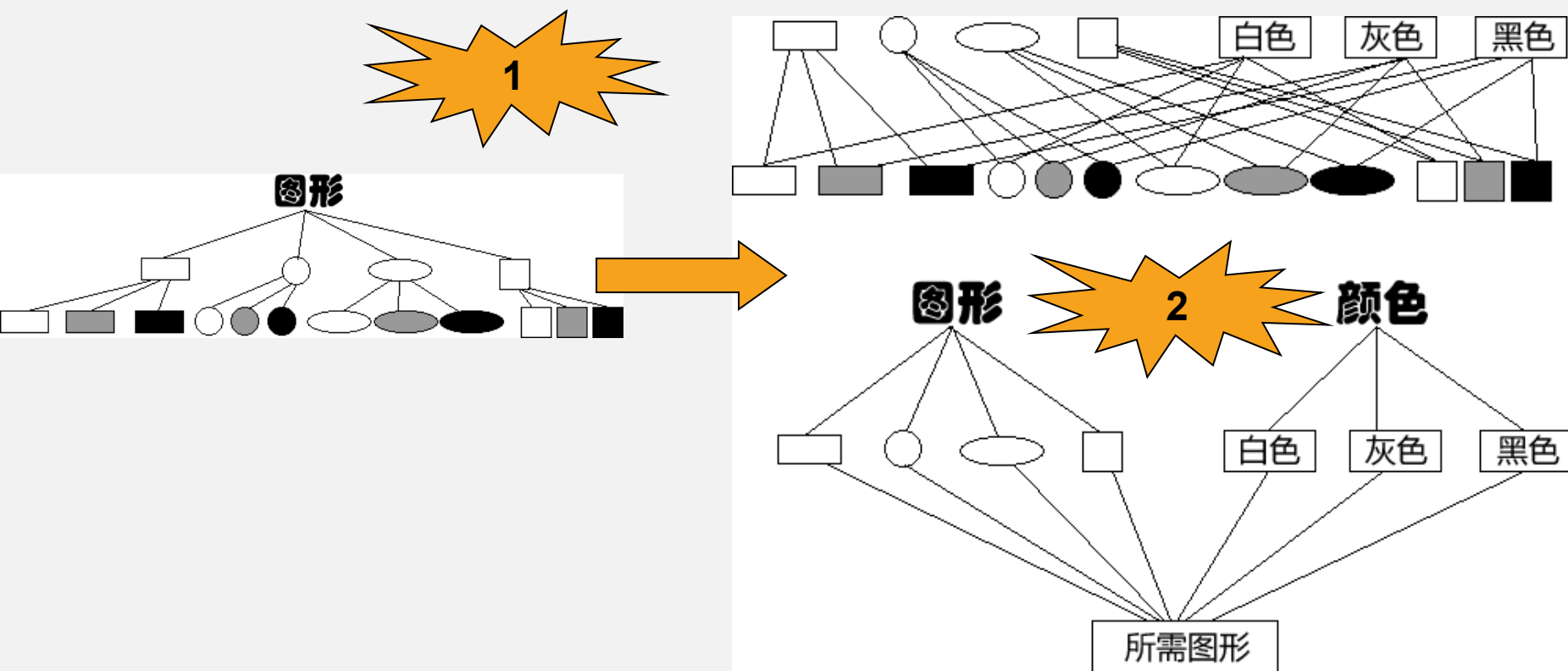
适配器模式

- 模式结构
 - 对象适配器



桥接模式

• 模式动机

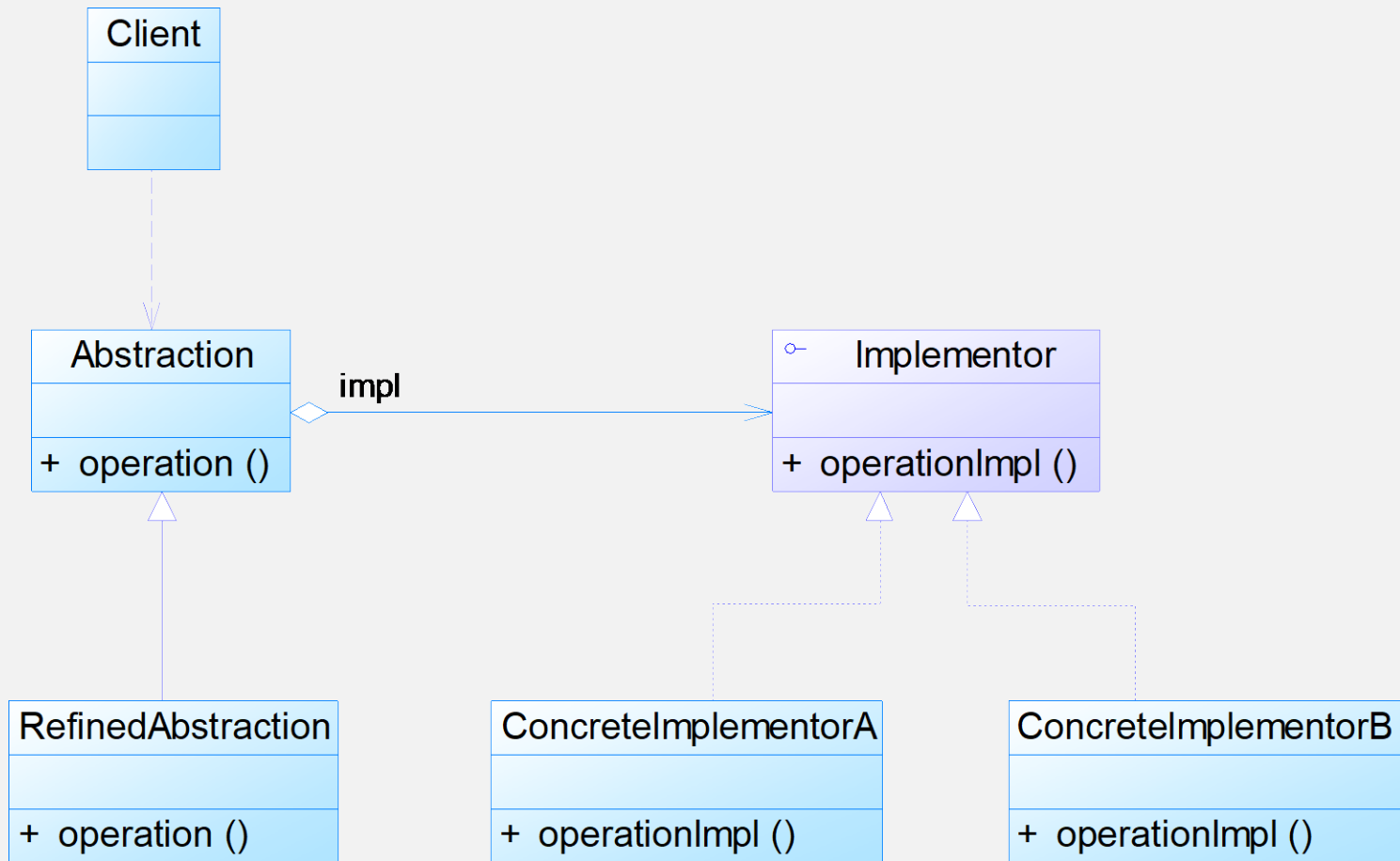


桥接模式

- 模式定义
 - 桥接模式(Bridge Pattern): 将抽象部分与它的实现部分分离, 使它们都可以独立地变化。它是一种对象结构型模式, 又称为柄体(Handle and Body)模式或接口(Interface)模式。
 - Bridge Pattern: Decouple an abstraction from its implementation so that the two can vary independently.

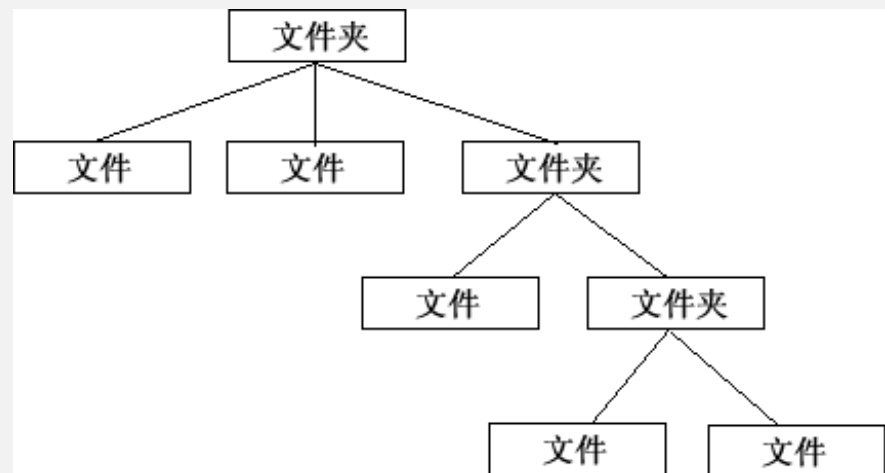
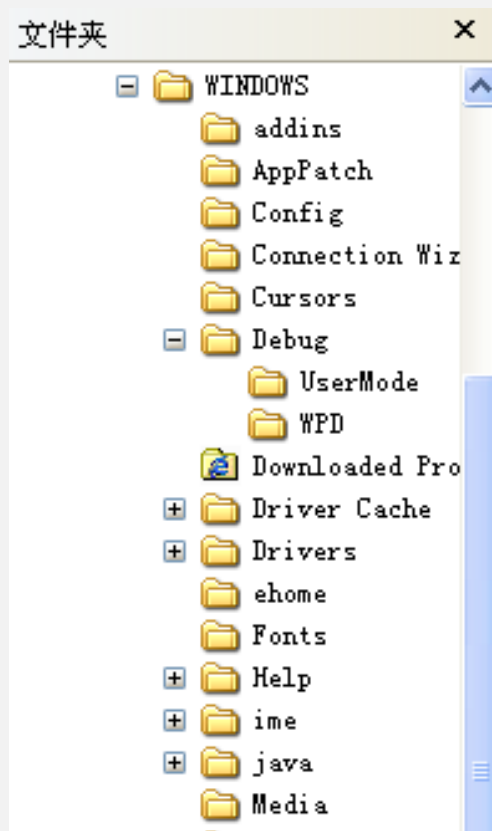
桥接模式

- 模式结构



组合模式

• 模式动机

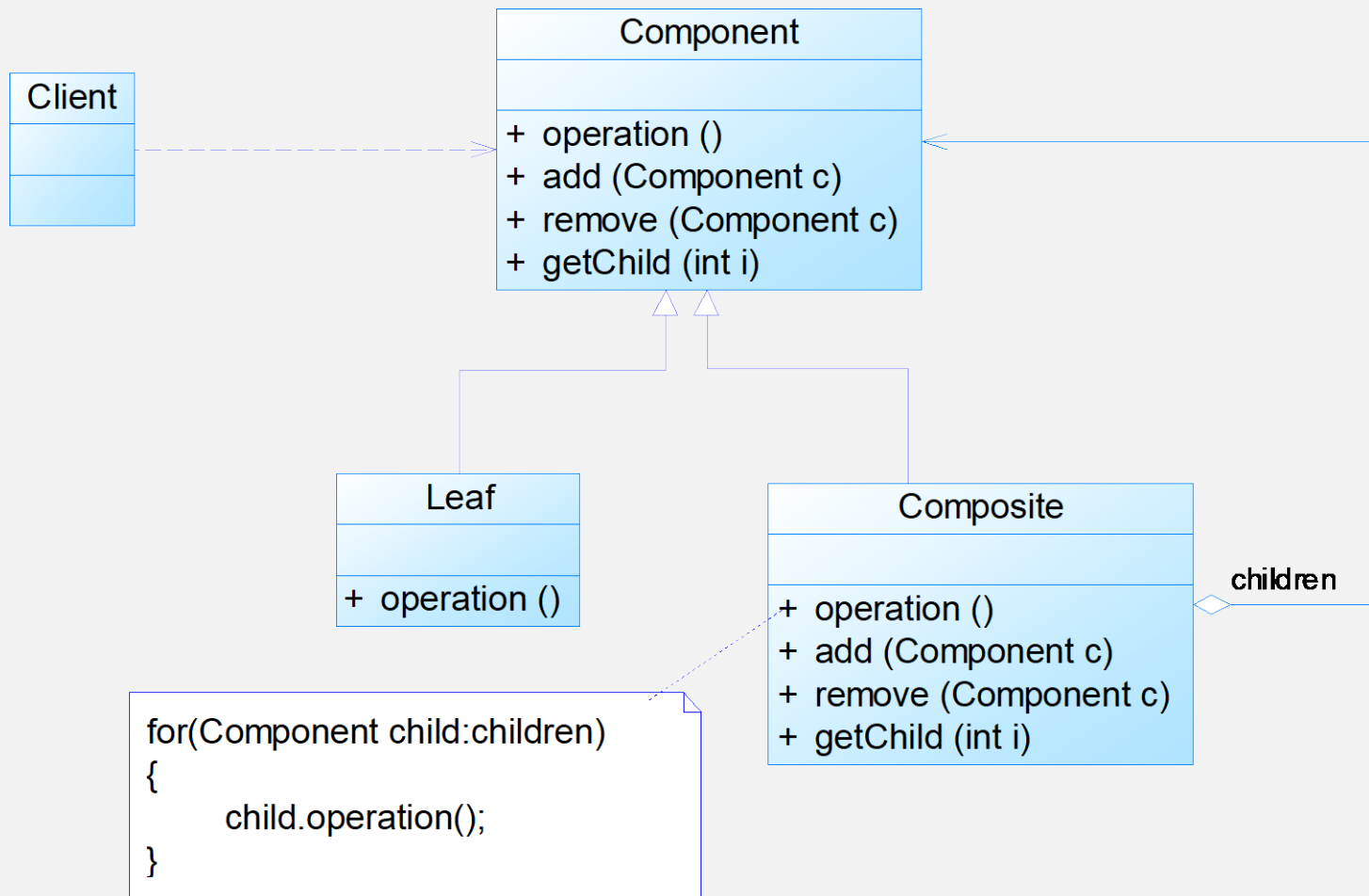


组合模式

- 模式定义
 - 组合模式(Composite Pattern): 组合多个对象形成树形结构以表示“整体-部分”的结构层次。组合模式对单个对象（即叶子对象）和组合对象（即容器对象）的使用具有一致性。
 - 组合模式又可以称为“整体-部分”(Part-Whole)模式，属于对象的结构模式，它将对象组织到树结构中，可以用来描述整体与部分的关系。
 - Composite Pattern: Compose objects into tree structures to represent part-whole hierarchies. Composite lets clients treat individual objects and compositions of objects uniformly.

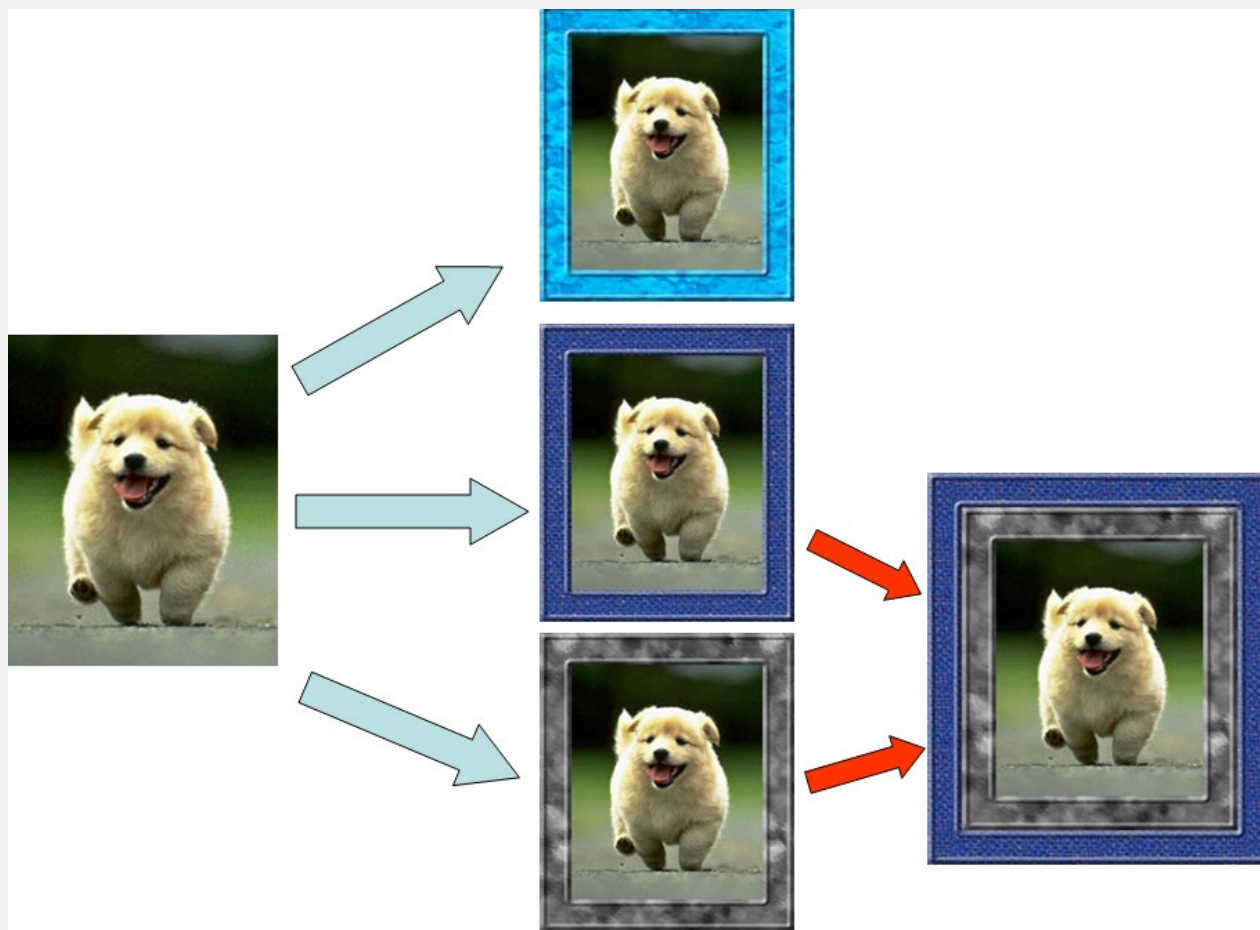
组合模式

- 模式结构



装饰模式

- 模式动机

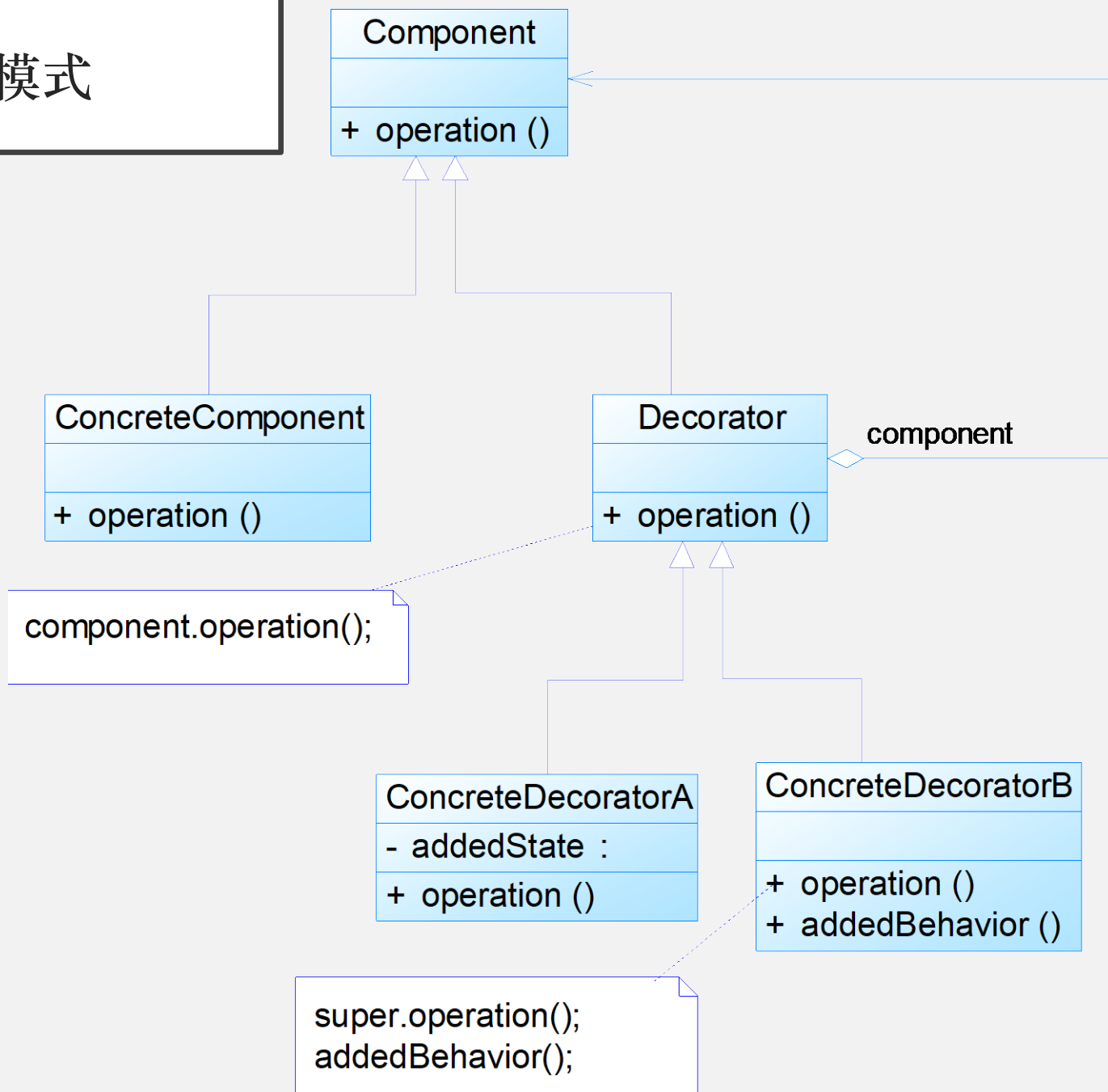


装饰模式

- 模式定义
 - 装饰模式(Decorator Pattern)：动态地给一个对象增加一些额外的职责 (Responsibility)，就增加对象功能来说，装饰模式比生成子类实现更为灵活。其别名也可以称为包装器(Wrapper)，与适配器模式的别名相同，但它们适用于不同的场合。根据翻译的不同，装饰模式也有人称之为“油漆工模式”，它是一种对象结构型模式。
 - Decorator Pattern: Attach additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality.

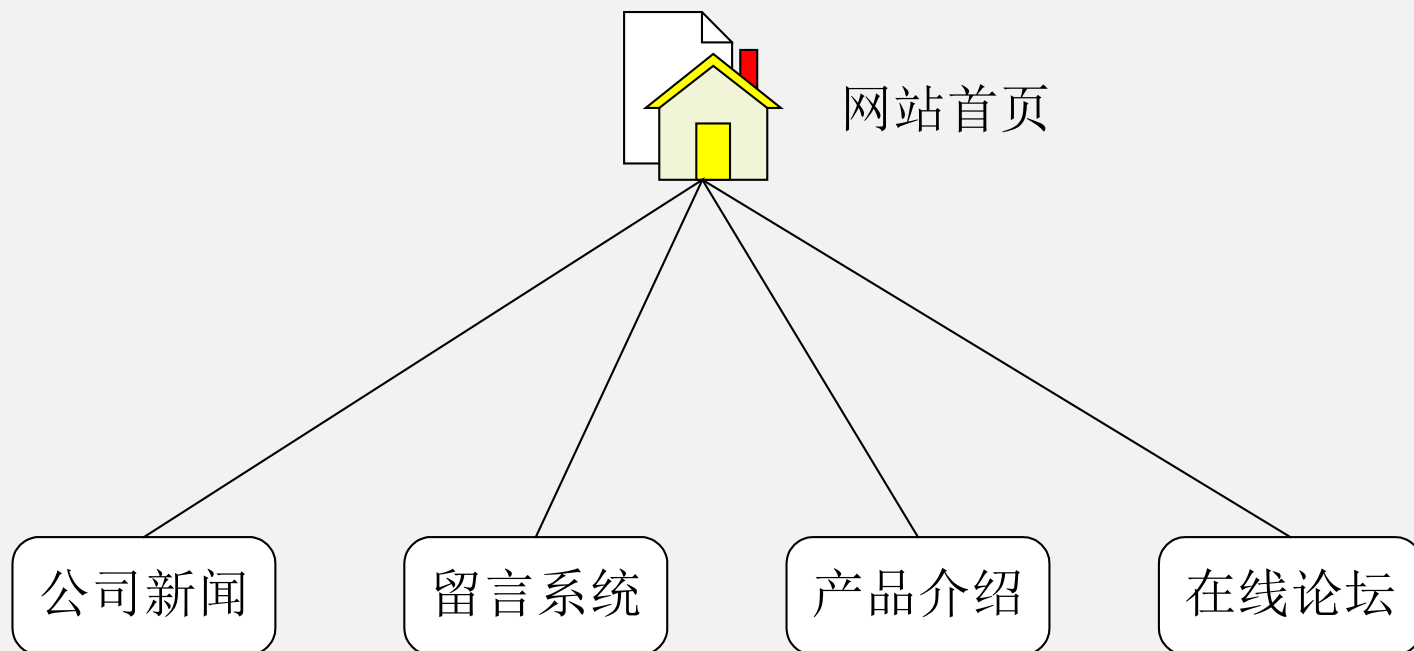
装饰模式

- 模式结构



外观模式

- 模式动机

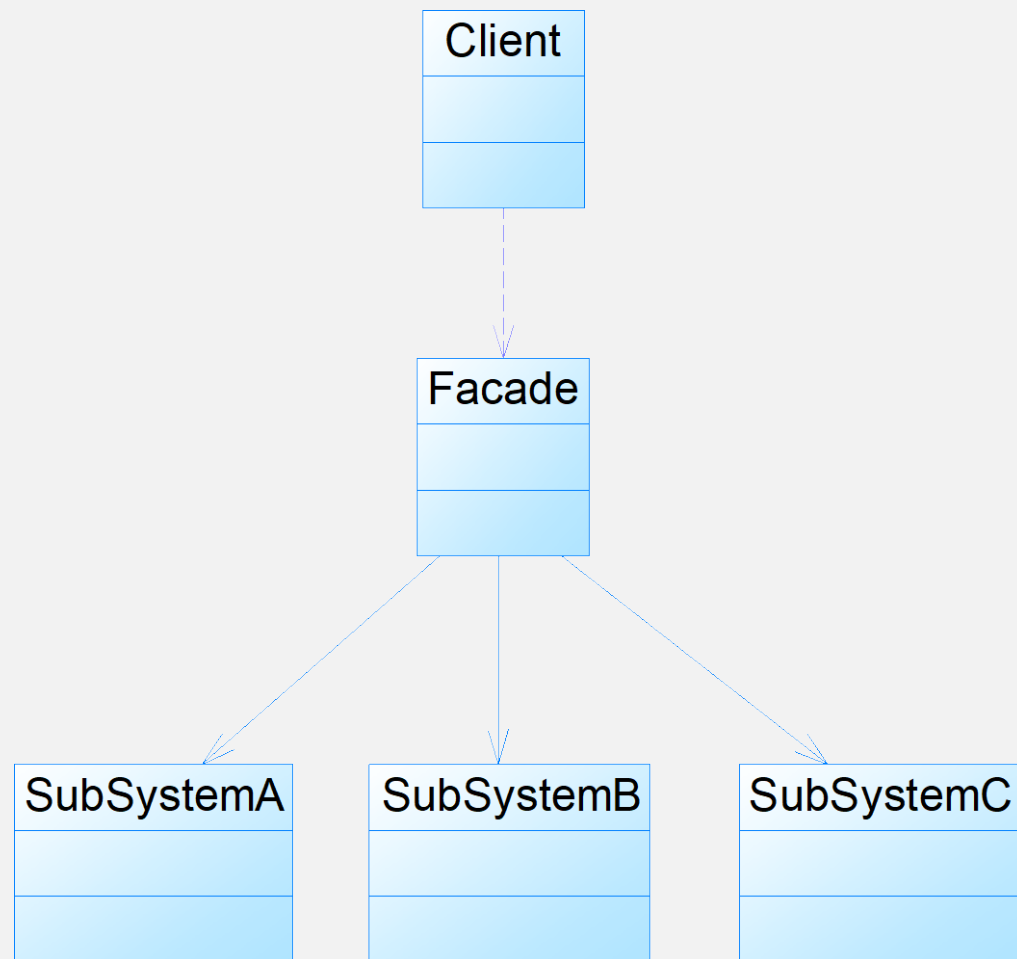
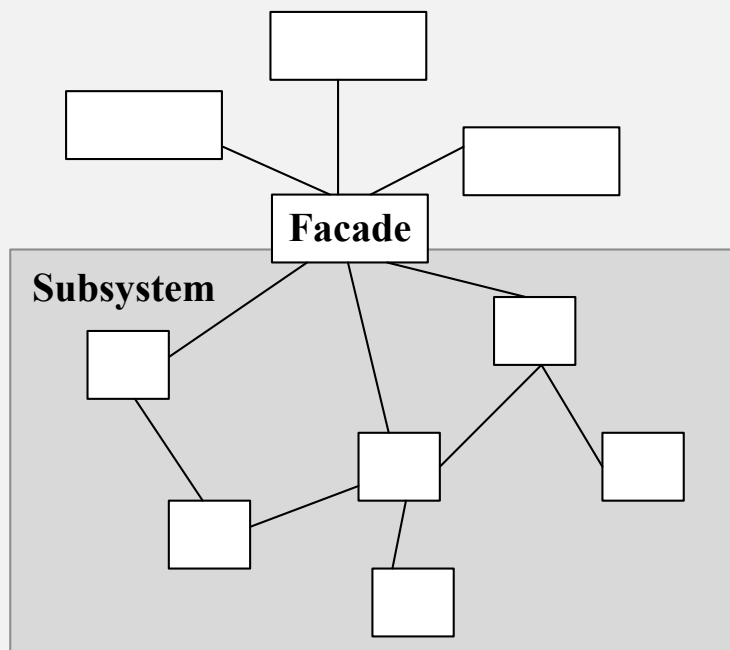


外观模式

- 模式定义
 - 外观模式(Facade Pattern): 外部与一个子系统的通信必须通过一个统一的外观对象进行, 为子系统的一组接口提供一个一致的界面, 外观模式定义了一个高层接口, 这个接口使得这一子系统更加容易使用。外观模式又称为门面模式, 它是一种对象结构型模式。
 - Facade Pattern: Provide a unified interface to a set of interfaces in a subsystem. Facade defines a higher-level interface that makes the subsystem easier to use.

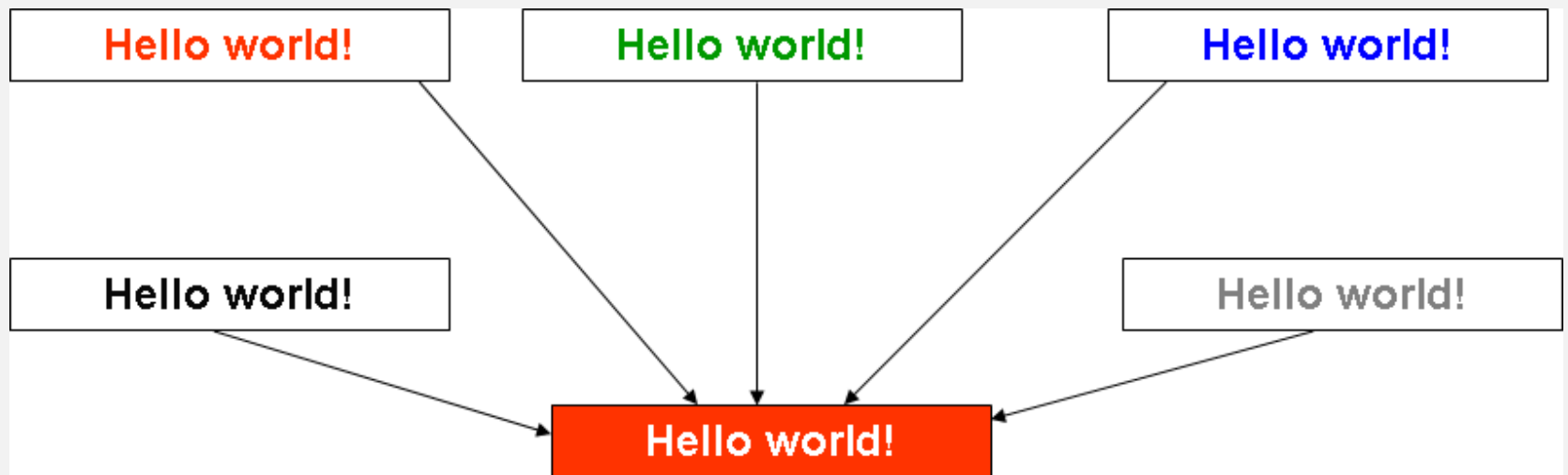
外观模式

- 模式结构



享元模式

- 模式动机

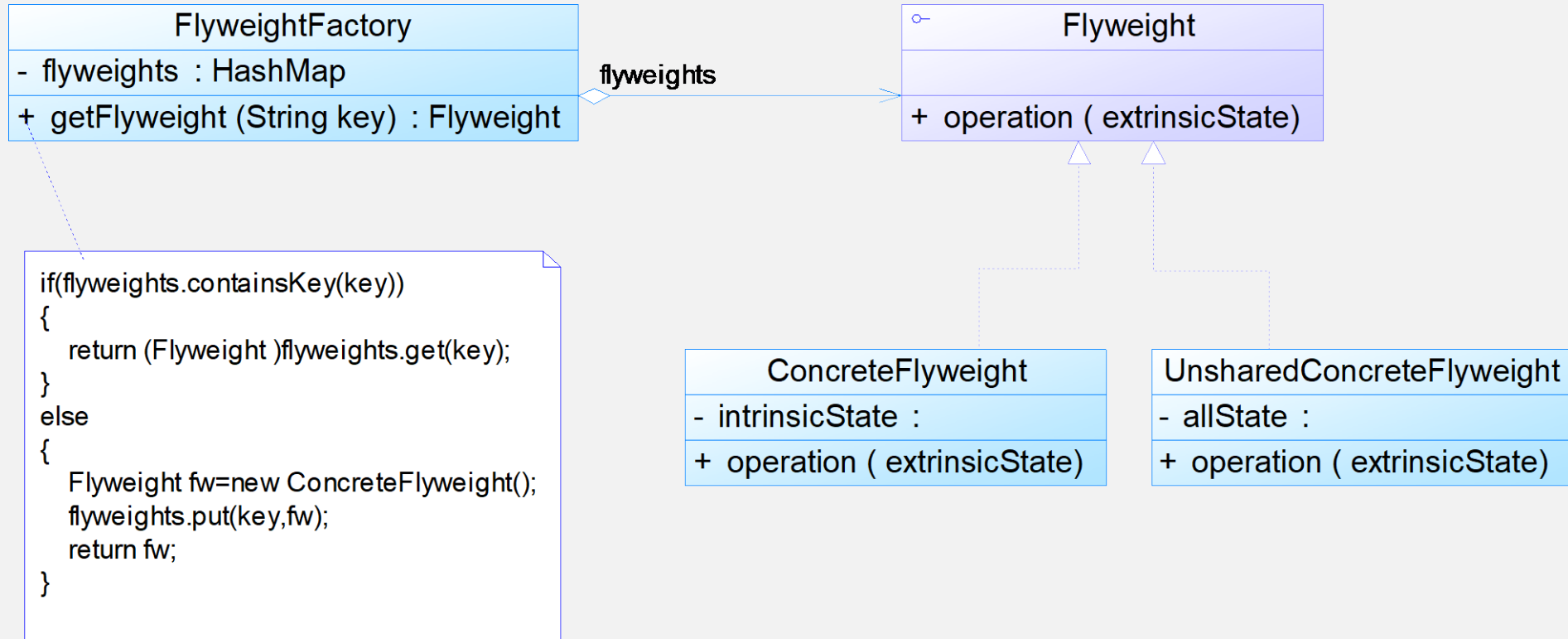


享元模式

- 模式定义
 - 享元模式(Flyweight Pattern): 运用共享技术有效地支持大量细粒度对象的复用。系统只使用少量的对象, 而这些对象都很相似, 状态变化很小, 可以实现对象的多次复用。由于享元模式要求能够共享的对象必须是细粒度对象, 因此它又称为轻量级模式, 它是一种对象结构型模式。
 - Flyweight Pattern: Use sharing to support large numbers of fine-grained objects efficiently.

享元模式

- 模式结构



代理模式

- 模式动机



小图片

代理



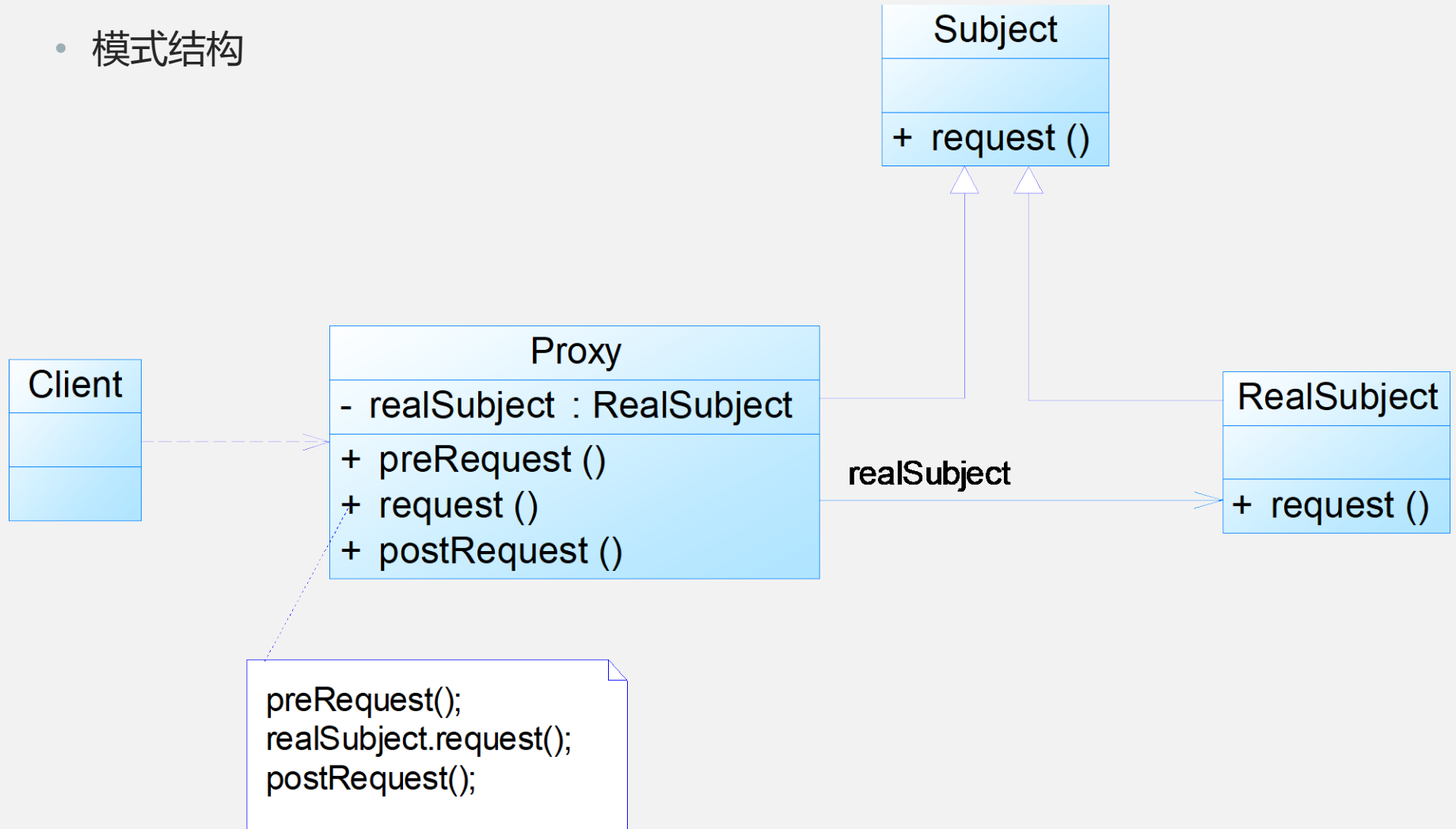
大图片

代理模式

- 模式定义
 - 代理模式(Proxy Pattern)：给某一个对象提供一个代理，并由代理对象控制对原对象的引用。代理模式的英文叫做Proxy或Surrogate，它是一种对象结构型模式。
 - Proxy Pattern: Provide a surrogate or placeholder for another object to control access to it.

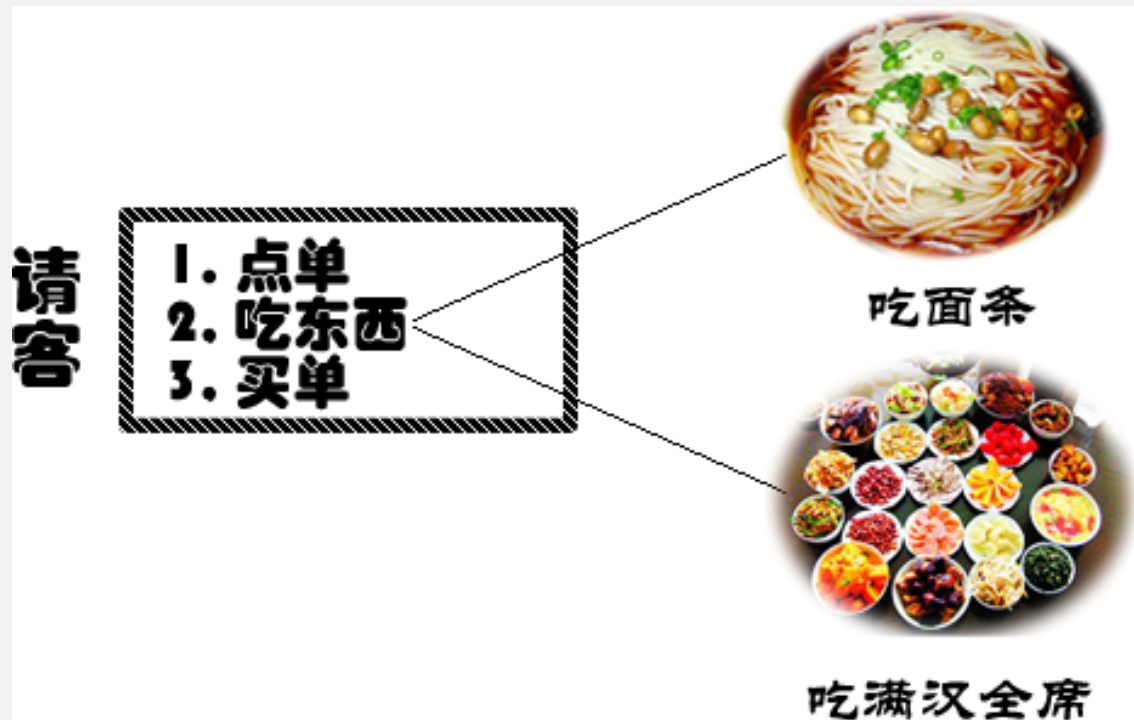
代理模式

- 模式结构



模板方法模式

- 模式动机

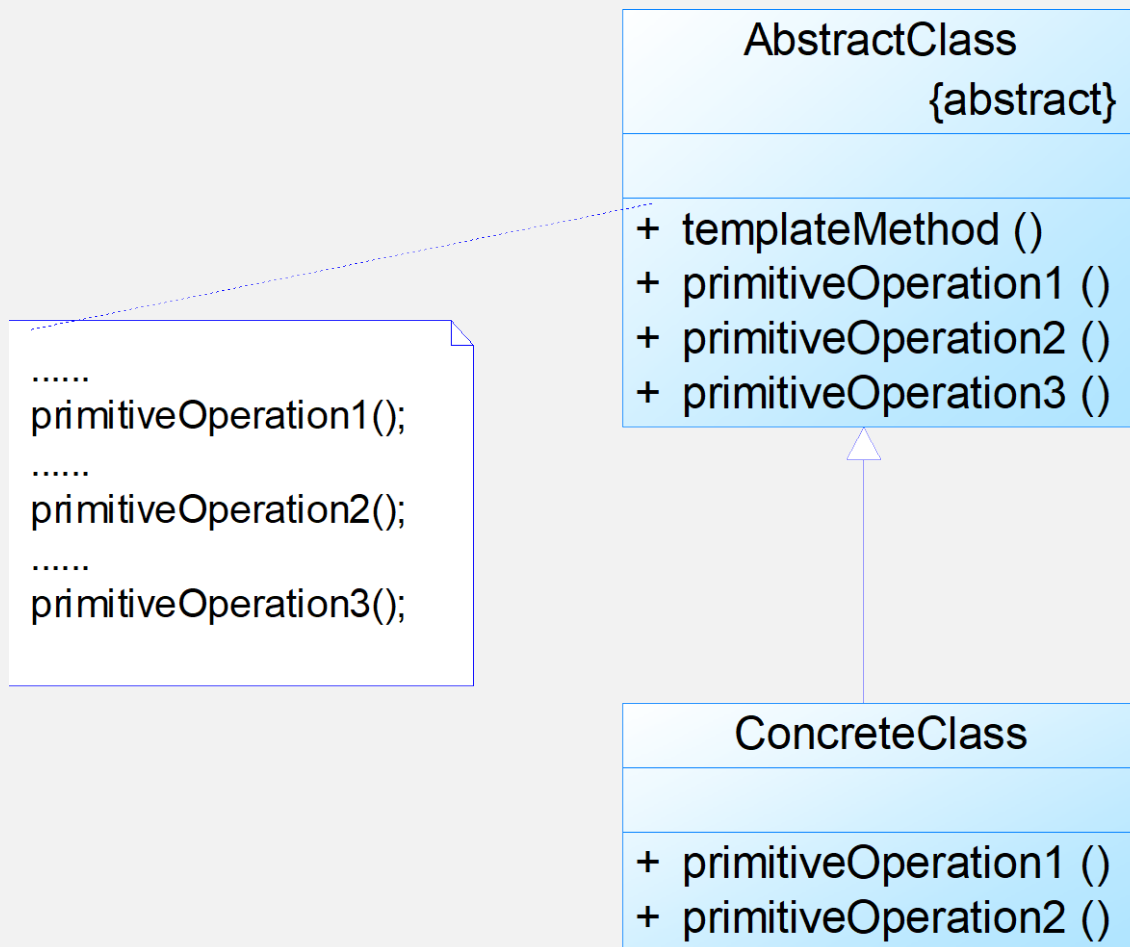


模板方法模式

- 模式定义
 - 模板方法模式(Template Method Pattern): 定义一个操作中**算法的骨架**, 而将一些步骤**延迟到子类**中, 模板方法使得子类**可以不改变一个算法的结构即可重定义该算法的某些特定步骤**。模板方法是一种**类行为型**模式。
 - Template Method Pattern: Define **the skeleton of an algorithm** in an operation, **deferring some steps to subclasses**. Template Method lets subclasses **redefine certain steps** of an algorithm **without changing the algorithm's structure**.

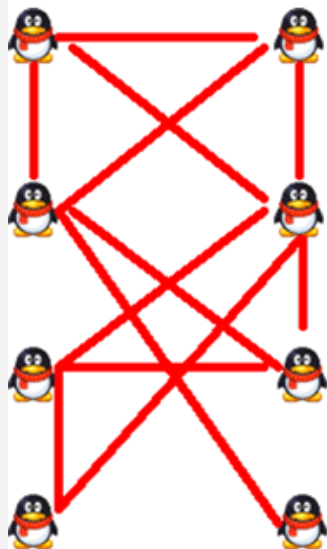
模板方法模式

- 模式结构



中介者模式

• 模式动机

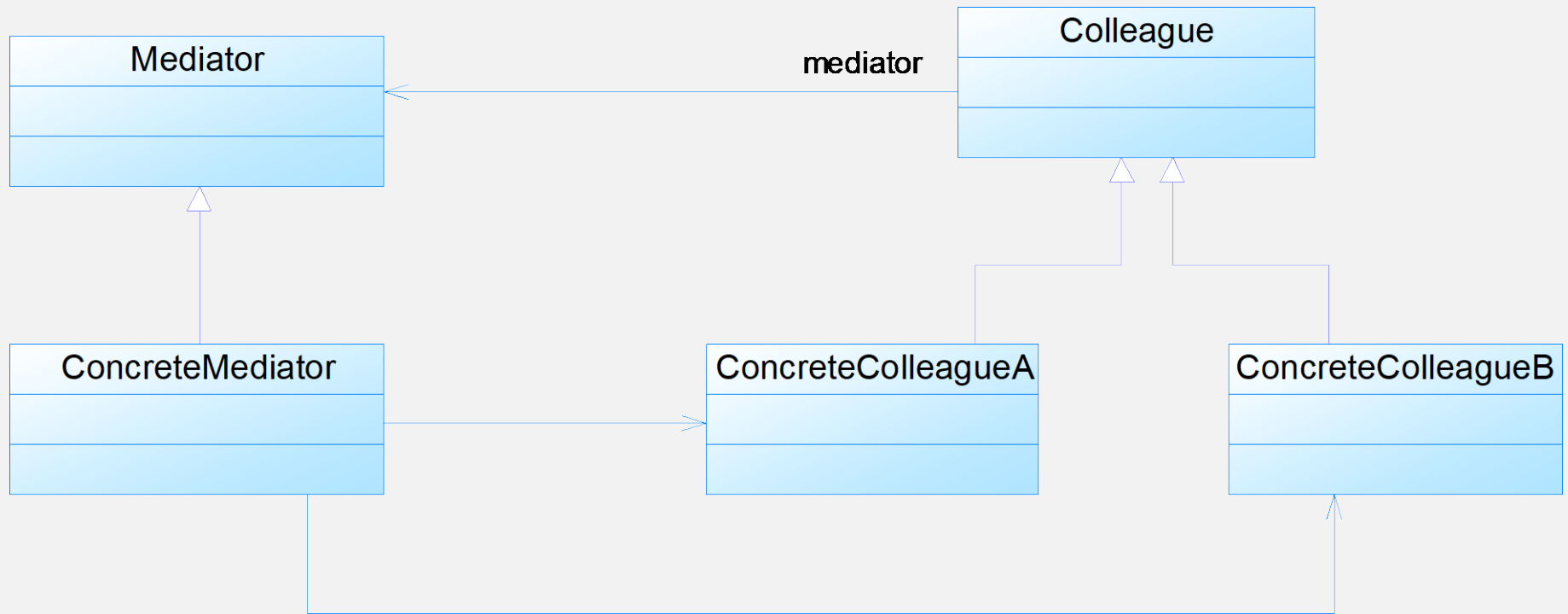


中介者模式

- 模式定义
 - 中介者模式(Mediator Pattern)定义：用一个中介对象来封装一系列的对象交互，中介者使各对象不需要显式地相互引用，从而使其耦合松散，而且可以独立地改变它们之间的交互。中介者模式又称为调停者模式，它是一种对象行为型模式。
 - Mediator Pattern: Define an object that encapsulates how a set of objects interact. Mediator promotes loose coupling by keeping objects from referring to each other explicitly, and it lets you vary their interaction independently.

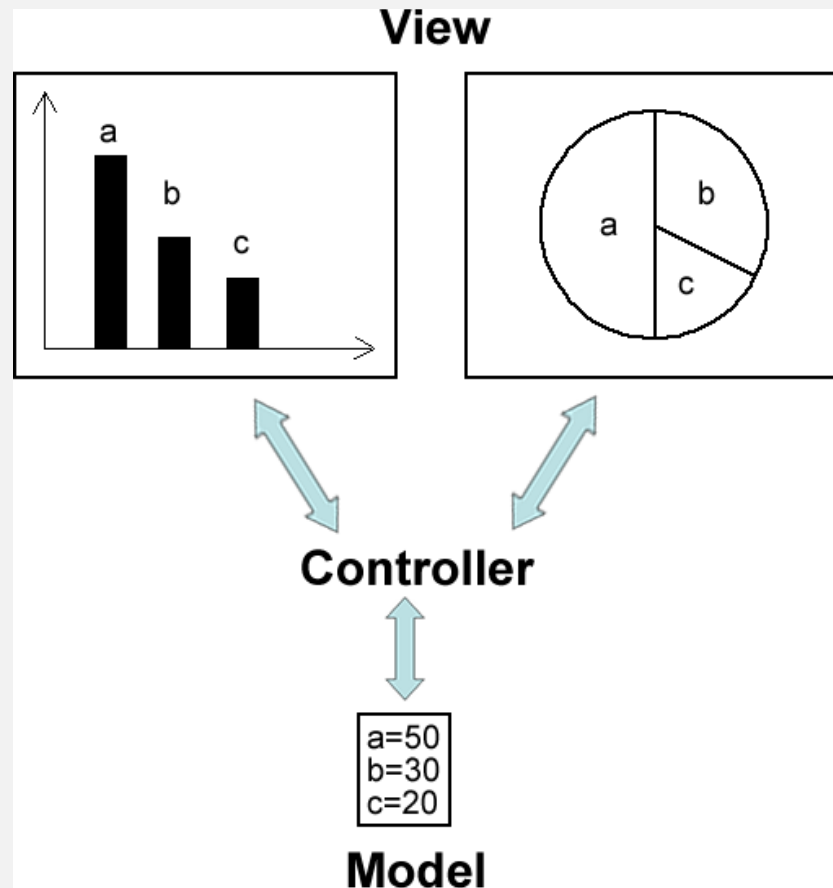
中介者模式

- 模式结构



观察者模式

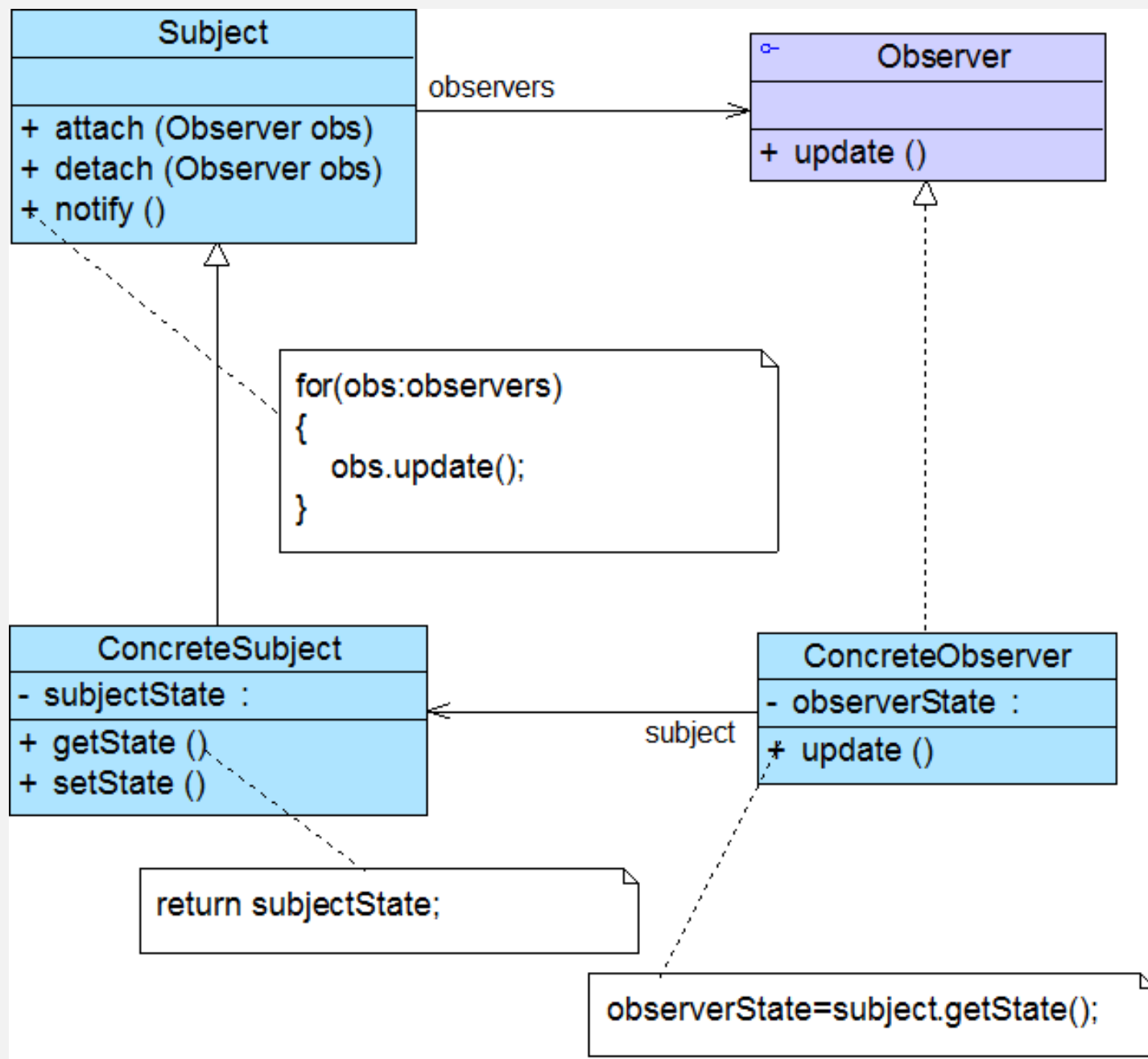
- 模式动机



观察者模式

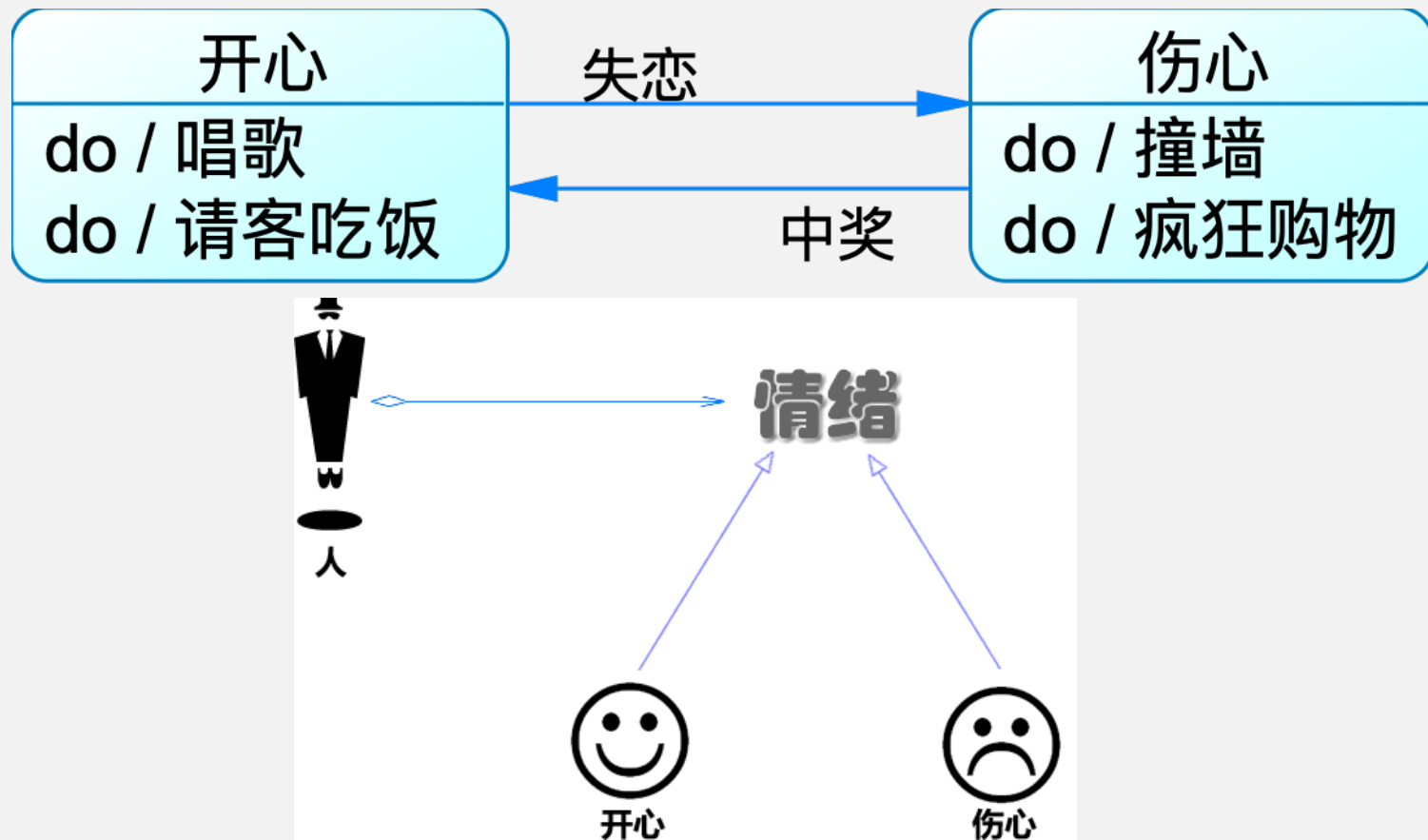
- 模式定义
 - 观察者模式(Observer Pattern): 定义对象间的一种一对多依赖关系, 使得每当一个对象状态发生改变时, 其相关依赖对象皆得到通知并被自动更新。观察者模式又叫做发布-订阅 (Publish/Subscribe) 模式、模型-视图 (Model/View) 模式、源-监听器 (Source/Listener) 模式或从属者 (Dependents) 模式。观察者模式是一种对象行为型模式。
 - Observer Pattern: Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.

- 模式结构



状态模式

- 模式动机

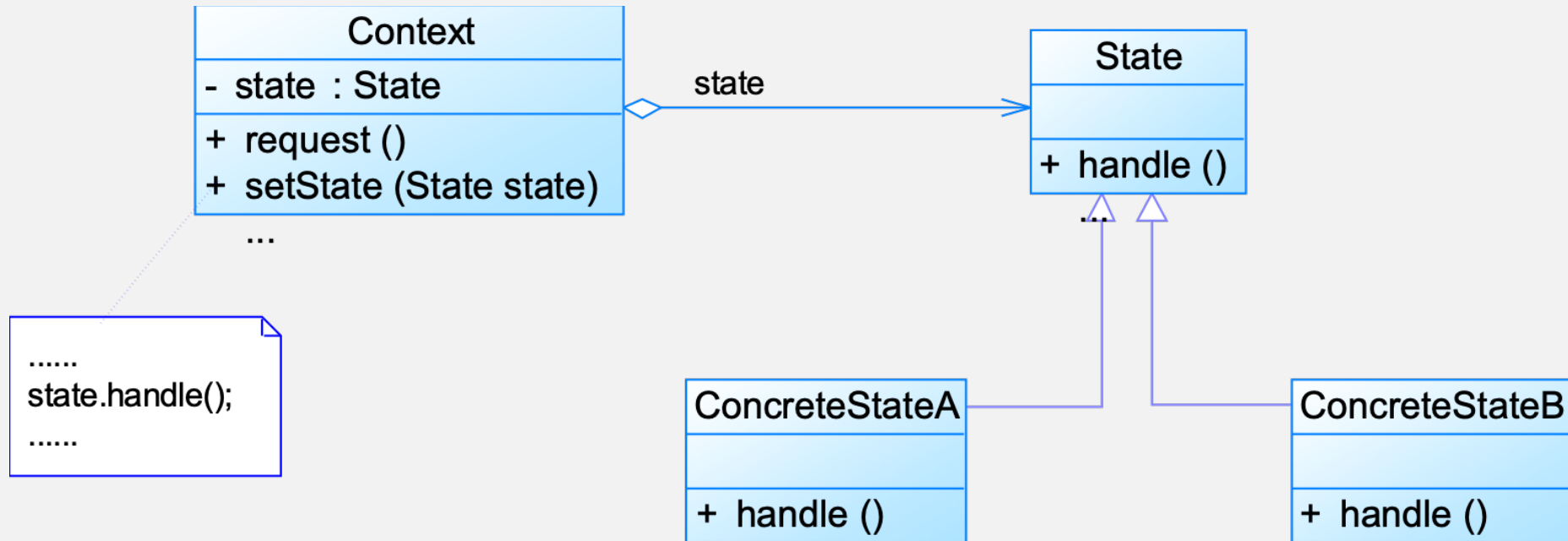


状态模式

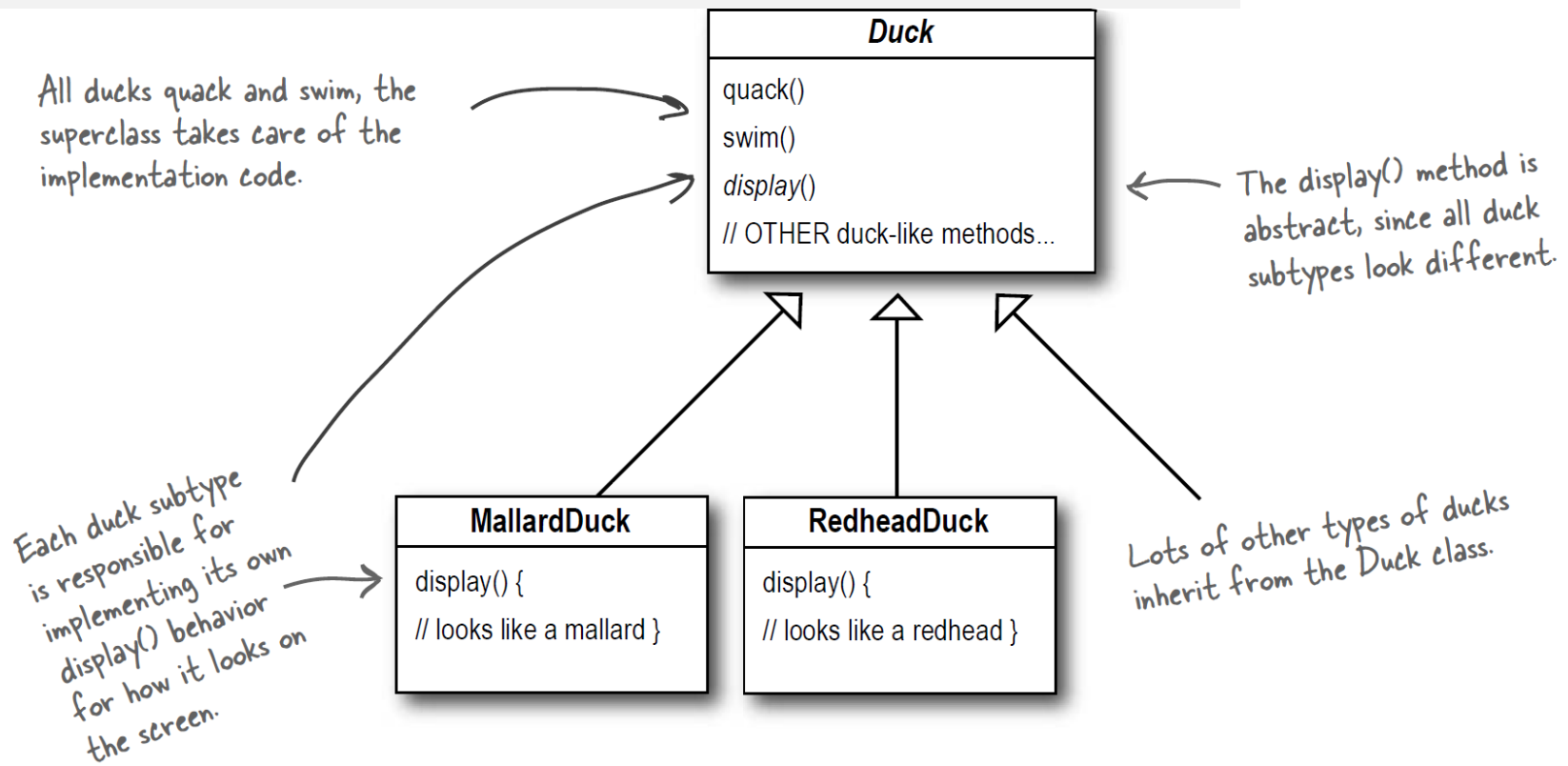
- 模式定义
 - 状态模式(State Pattern)：允许一个对象在其内部状态改变时改变它的行为，对象看起来似乎修改了它的类。其别名为状态对象(Objects for States)，状态模式是一种对象行为型模式。
 - State Pattern: Allow an object to alter its behavior when its internal state changes. The object will appear to change its class.

状态模式

- 模式结构



策略模式

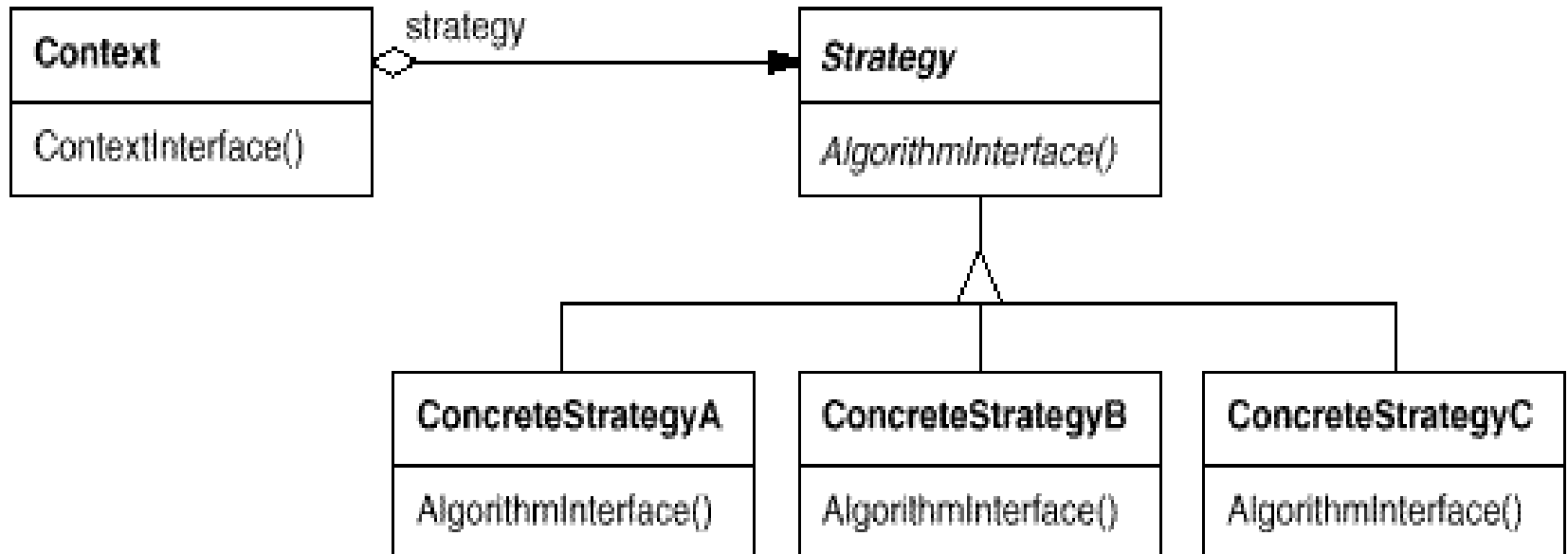


策略模式

- Name:
 - Strategy
- Intent:
 - Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it.
- Also known as:
 - Policy

策略模式

- The Strategy Pattern defines a family of algorithms, encapsulates each one, and makes them interchangeable. Strategy lets the algorithm vary independently from clients that use it.



设计模式与类库框架

Q: If design patterns are so great, why can't someone build a library of them so I don't have to?

A: Design patterns are higher level than libraries. Design patterns tell us how to structure classes and objects to solve certain problems and it is our job to adapt those designs to fit our particular application.

Q: Aren't libraries and frameworks also design patterns?

A: Frameworks and libraries are not design patterns; they provide specific implementations that we link into our code. Sometimes, however, libraries and frameworks make use of design patterns in their implementations. That's great, because once you understand design patterns, you'll more quickly understand APIs that are structured around design patterns.

设计原则

- 对于面向对象的软件系统设计来说，在支持可维护性的同时，需要提高系统的可复用性。
- 软件的复用可以提高软件的开发效率，提高软件质量，节约开发成本，恰当的复用还可以改善系统的可维护性。
- 单一职责原则要求在软件系统中，一个类只负责一个功能领域中的相应职责。
- 开闭原则要求一个软件实体应当对扩展开放，对修改关闭，即在不修改源代码的基础上扩展一个系统的行为。
- 里氏代换原则可以通俗表述为在软件中如果能够使用基类对象，那么一定能够使用其子类对象。

设计原则

- 依赖倒转原则要求抽象不应该依赖于细节，细节应该依赖于抽象；要针对接口编程，不要针对实现编程。
- 接口隔离原则要求客户端不应该依赖那些它不需要的接口，即将一些大的接口细化成一些小的接口供客户端使用。
- 合成复用原则要求复用时尽量使用对象组合，而不使用继承。
- 迪米特法则要求一个软件实体应当尽可能少的与其他实体发生相互作用。

设计原则

- 目标：开闭原则
- 指导：最小知识原则
- 基础：单一职责原则、可变性封装原则
- 实现：依赖倒转原则、合成复用原则、里氏代换原则、接口隔离原则